

Funzioni

Funzioni in Python

Come creare blocchi di codice riutilizzabili per non ripetere sempre le stesse cose.

Perché usare le funzioni?

Immagina di dover calcolare la media dei voti tre volte in un programma. Senza funzioni, dovresti scrivere la stessa formula tre volte. Se poi vuoi cambiare come si calcola la media, devi ricordarti di cambiarla in tutti e tre i posti.

Con le funzioni, scrivi il calcolo **una volta sola**, gli dai un nome, e lo riusi quante volte vuoi. Se cambia qualcosa, modifichi solo la funzione.

Creare una funzione

Si usa la parola chiave `def` (abbreviazione di "define", definire):

```
def saluta():  
    print("Ciao!")
```

Questa funzione si chiama `saluta` e, quando viene eseguita, stampa "Ciao!". Ma attenzione: definire la funzione non la esegue ancora.

Chiamare una funzione

Per eseguire la funzione, la "chiami" scrivendone il nome seguito dalle parentesi:

```
saluta() # esegue la funzione → stampa "Ciao!"  
saluta() # puoi richiamarla quante volte vuoi  
saluta()
```

Passare informazioni: i parametri

Le funzioni diventano molto più utili quando possono ricevere informazioni dall'esterno. Queste informazioni si chiamano **parametri**:

```
def saluta(nome):  
    print(f"Ciao, {nome}!")  
  
saluta("Alice")    # Ciao, Alice!  
saluta("Bob")      # Ciao, Bob!  
saluta("Carlo")    # Ciao, Carlo!
```

La stessa funzione si comporta in modo diverso in base a cosa le passi. Puoi avere più parametri:

```
def somma(a, b):  
    print(a + b)  
  
somma(3, 5)        # 8  
somma(10, 20)      # 30
```

Restituire un risultato: `return`

Finora le funzioni hanno solo stampato qualcosa. Ma spesso vuoi che la funzione **calcoli qualcosa e ti restituisca il risultato**, in modo da poterlo usare dopo.

Si usa la parola chiave `return` :

```
def somma(a, b):  
    return a + b  
  
risultato = somma(3, 5)  
print(risultato)          # 8  
print(somma(10, 20) * 2) # 60 - puoi usarlo direttamente nelle espressioni
```

Quando Python incontra `return` , esce dalla funzione e consegna il valore. Se una funzione non ha `return` , restituisce `None` automaticamente.

Valori predefiniti per i parametri

Puoi assegnare un valore di default a un parametro. Se non viene passato, usa quel valore:

```
def saluta(nome, saluto="Ciao"):
    print(f"{saluto}, {nome}!")

saluta("Alice")          # Ciao, Alice! – usa il default
saluta("Bob", "Buongiorno") # Buongiorno, Bob! – sovrascrive il default
```

I parametri con valore di default devono sempre venire **dopo** quelli senza.

Passare argomenti per nome

Puoi specificare il nome di ogni parametro quando chiami la funzione. L'ordine diventa irrilevante:

```
def descrivi_persona(nome, eta, citta):
    print(f"{nome} ha {eta} anni e vive a {citta}.")

# Ordine normale
descrivi_persona("Alice", 16, "Roma")

# Con i nomi – l'ordine non importa più
descrivi_persona(eta=16, citta="Roma", nome="Alice")
```

Restituire più valori

Una funzione può restituire più di un valore contemporaneamente:

```
def min_max(neri):
    return min(neri), max(neri)

piccolo, grande = min_max([3, 1, 4, 1, 5, 9])
print(piccolo) # 1
print(grande) # 9
```

Esempio pratico

```
def calcola_media(voti):
    if len(voti) == 0:
        return 0
    return sum(voti) / len(voti)

def classifica_voto(media):
    if media >= 9:
        return "Ottimo"
    elif media >= 7:
        return "Buono"
    elif media >= 6:
        return "Sufficiente"
    else:
        return "Insufficiente"

voti_alice = [8, 7, 9, 6, 10]
media = calcola_media(voti_alice)
giudizio = classifica_voto(media)

print(f"Media: {media:.1f} - {giudizio}")
# Media: 8.0 - Buono
```

Nota come il programma principale è facile da leggere: capisce cosa fa senza dover sapere come funziona ogni funzione internamente.

Funzioni lambda

Funzioni piccole e veloci da scrivere in una sola riga.

Cosa sono le funzioni lambda?

Quando hai bisogno di una funzione semplice, che fa solo una cosa, e non vuoi sprecare righe di codice per definirla con `def`, puoi usare una **funzione lambda**.

È come una funzione normale, ma compressa in una singola riga, senza nome e senza `return` esplicito.

La forma base

```
lambda parametri: espressione
```

Confronta una funzione normale con la sua versione lambda:

```
# Funzione normale
def raddoppia(x):
    return x * 2

# La stessa cosa come lambda
raddoppia_lambda = lambda x: x * 2

print(raddoppia(5))          # 10
print(raddoppia_lambda(5))   # 10
```

Puoi anche avere più parametri:

```
somma = lambda a, b: a + b
print(somma(3, 5))  # 8
```

Quando usare le lambda?

Le lambda sono utili quando devi passare una "piccola funzione" come argomento a un'altra funzione. Il caso più comune è con `sorted()`, `map()` e `filter()`.

Ordinare una lista di oggetti complessi

Se hai una lista di tuple (coppie di valori) e vuoi ordinarla in base al secondo elemento, la lambda è perfetta:

```

studenti = [
    ("Alice", 16),
    ("Bob", 15),
    ("Carlo", 17)
]

# Ordina per età (il secondo elemento di ogni tupla)
ordinati = sorted(studenti, key=lambda s: s[1])
print(ordinati)
# [('Bob', 15), ('Alice', 16), ('Carlo', 17)]

```

Trasformare ogni elemento di una lista con `map()`

`map()` applica una funzione a ogni elemento di una sequenza:

```

numeri = [1, 2, 3, 4, 5]
quadrati = list(map(lambda x: x ** 2, numeri))
print(quadrati) # [1, 4, 9, 16, 25]

```

Equivale a: "per ogni numero nella lista, calcola il suo quadrato".

Filtrare elementi con `filter()`

`filter()` tiene solo gli elementi per cui la funzione restituisce `True` :

```

numeri = [1, 2, 3, 4, 5, 6, 7, 8]
pari = list(filter(lambda x: x % 2 == 0, numeri))
print(pari) # [2, 4, 6, 8]

```

Equivale a: "tieni solo i numeri che, divisi per 2, danno resto zero".

Limiti delle lambda

Le lambda sono semplici per design: possono contenere solo **una singola espressione**. Non puoi usare `if...elif...else` su più righe, cicli, o più istruzioni.

Quando la logica diventa più complessa di una riga, usa sempre una funzione normale con `def` :

```
# Questo va bene con lambda:
doppio = lambda x: x * 2

# Questo è troppo complesso – usa def:
def classifica(voto):
    if voto >= 9:
        return "Ottimo"
    elif voto >= 6:
        return "Sufficiente"
    else:
        return "Insufficiente"
```

Funzioni ricorsive

Funzioni che risolvono un problema chiamando se stesse.

Cos'è la ricorsione?

Una funzione ricorsiva è una funzione che, per risolvere un problema, **chiama se stessa** con una versione più piccola dello stesso problema.

Un'analogia: immagina di dover smontare una bambola matrioska (quelle russe una dentro l'altra). Come la apri? Apri la prima, trovi un'altra matrioska più piccola, la apri, trovi un'altra ancora più piccola... finché non trovi quella tinyissima che non si apre più. Quella è il **caso base**.

Le due parti fondamentali

Ogni funzione ricorsiva deve avere:

1. **Il caso base** — la condizione che fa fermare la ricorsione
2. **Il caso ricorsivo** — dove la funzione chiama se stessa con un problema più piccolo

Senza il caso base, la funzione si chiamerebbe all'infinito e il programma si bloccherebbe con un errore.

Esempio classico: il fattoriale

Il fattoriale di un numero (scritto con il simbolo **!**) è il prodotto di tutti i numeri interi da 1 fino a quel numero:

```
5! = 5 × 4 × 3 × 2 × 1 = 120
3! = 3 × 2 × 1 = 6
1! = 1
```

Si può anche scrivere così: **5! = 5 × 4!** . E **4! = 4 × 3!** . Vedi il pattern? Il problema si riduce sempre.

```
def fattoriale(n):
    # Caso base: se n è 0 o 1, il fattoriale è 1
    if n == 0 or n == 1:
        return 1
    # Caso ricorsivo: n * fattoriale del numero precedente
    return n * fattoriale(n - 1)

print(fattoriale(5)) # 120
print(fattoriale(3)) # 6
```

Ecco cosa succede passo per passo quando chiami **fattoriale(5)** :

```
fattoriale(5)
→ 5 × fattoriale(4)
    → 4 × fattoriale(3)
        → 3 × fattoriale(2)
            → 2 × fattoriale(1)
                → 1 ← caso base, si ferma!
```

Poi si torna indietro calcolando: **2×1=2** , **3×2=6** , **4×6=24** , **5×24=120** .

Esempio: la successione di Fibonacci

La successione di Fibonacci è famosa in matematica e in natura: **0, 1, 1, 2, 3, 5, 8, 13, 21, ...**

Ogni numero è la somma dei due che lo precedono. Anche questo si presta alla ricorsione:


```
def fibonacci(n):  
    if n == 0:  
        return 0  
    if n == 1:  
        return 1  
    return fibonacci(n - 1) + fibonacci(n - 2)  
  
for i in range(8):  
    print(fibonacci(i), end=" ")  
# 0 1 1 2 3 5 8 13
```

Un limite importante

Python può gestire al massimo circa 1000 chiamate ricorsive in profondità. Se superi questo limite, ottieni un `RecursionError`. Per questo la ricorsione non è sempre la scelta migliore per problemi con numeri molto grandi.

Quando usare la ricorsione?

La ricorsione è utile quando il problema ha una struttura naturalmente "annidata" — cioè quando si può descrivere un problema grande come una versione più piccola dello stesso problema.

Per problemi semplici (come contare fino a 100, o scorrere una lista), usa i cicli `for` o `while`: sono più veloci e più facili da capire.

Scope delle variabili

Dove le variabili esistono e dove sono accessibili nel codice.

Cos'è lo scope?

Immagina di avere una variabile `x` dentro una funzione. Quella `x` esiste solo dentro quella funzione. Fuori non la puoi usare, come se non esistesse.

Questo concetto si chiama **scope** (o "ambito"): la zona del programma in cui una variabile è accessibile.

Variabili locali — dentro la funzione

Una variabile creata dentro una funzione è **locale**: vive solo per la durata di quella funzione e poi scompare.

```
def mia_funzione():  
    x = 10 # variabile locale: esiste solo dentro questa funzione  
    print(x)  
  
mia_funzione() # 10  
# print(x)      # NameError: 'x' non esiste qui fuori!
```

Questo è un comportamento voluto, non un difetto. Significa che le funzioni sono **indipendenti**: le variabili dentro una funzione non possono interferire con quelle fuori.

Variabili globali — fuori dalle funzioni

Una variabile definita fuori da qualsiasi funzione è **globale**: è visibile da tutta la parte principale del programma.

```
nome = "Alice" # variabile globale  
  
def saluta():  
    print(f"Ciao, {nome}!") # può leggere la variabile globale  
  
saluta() # Ciao, Alice!
```

Puoi leggere le variabili globali, ma non modificarle (di solito)

Una funzione può **leggere** una variabile globale, ma se prova a modificarla, Python crea una nuova variabile locale con lo stesso nome invece di cambiare quella globale:

```

contatore = 0

def incrementa():
    contatore = 1 # ATTENZIONE: crea una variabile locale, non modifica
    quella globale!
    print(f"Dentro la funzione: {contatore}")

incrementa()
print(f"Fuori dalla funzione: {contatore}") # ancora 0, non è cambiato!

```

Se vuoi davvero modificare una variabile globale dall'interno di una funzione, devi usare la parola chiave `global` :

```

contatore = 0

def incrementa():
    global contatore # ora si riferisce alla variabile globale
    contatore += 1

incrementa()
incrementa()
print(contatore) # 2

```

:::tip[Consiglio] Usare `global` è generalmente una cattiva abitudine. Rende il codice difficile da capire e da correggere. È meglio passare i valori come parametri e usare `return` per restituirli. :::

Variabili con lo stesso nome a livelli diversi

Se hai una variabile locale e una globale con lo stesso nome, dentro la funzione vince quella locale:

```

x = "globale"

def funzione():
    x = "locale"
    print(x) # locale – la variabile locale ha la precedenza

funzione()
print(x) # globale – fuori dalla funzione, x è ancora quella globale

```

Non vengono confuse: sono due variabili separate che condividono solo il nome.

Le funzioni predefinite di Python

Funzioni come `print`, `len`, `range`, `input` sono sempre disponibili ovunque nel codice. Python le cerca per ultime, ma le trova sempre.

Per questo non dovresti mai chiamare una tua variabile `print` o `list`: sovrascriveresti la funzione predefinita e poi non potresti più usarla!

```
# Da non fare mai!  
# list = [1, 2, 3]   – ora non puoi più usare list() come funzione  
# print = "ciao"    – ora print() non funziona più!
```